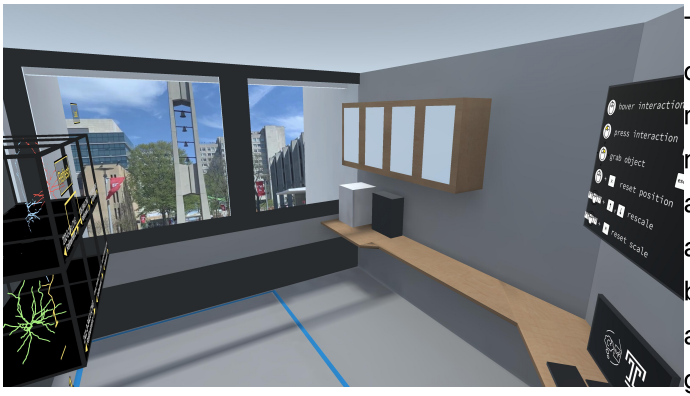
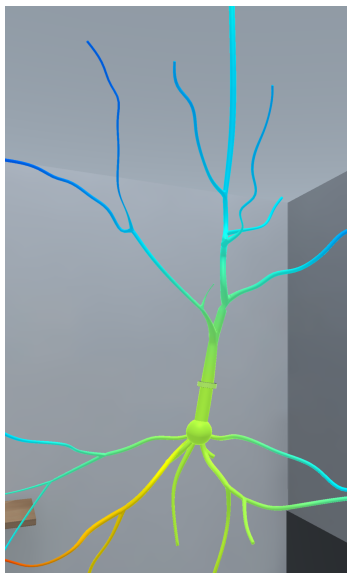
Neuro-VISOR

VISOR (Virtual Interactive Simulation Of Reality) is a research project developed by Temple University's Center for Computational Mathematics and Modeling (C2M2), College of Science and Technology. The overarching long-term vision of VISOR is to produce novel concepts and software that enable efficient immersed virtual reality (VR) visualization and real-time interaction with simulations of real-world processes described via principled mathematical equations. Unlike traditional high performance computing (HPC) applications, the philosophy of VISOR is that (a) the simulation runs while it is visualized in a virtual environment, and (b) the user can affect and modify the system state or its conditions at runtime, and the simulation reacts instantaneously and seamlessly.



The Neuro-VISOR project focuses on applications in computational neuroscience. Selected neurons retrieved as wire-frame geometries from the public neuron database NeuroMorpho have been augmented by surface meshes for 3D visualization and user interaction. These neurons are available to be placed in the virtual room, in which an efficient and robust numerical method approximates governing partial differential equation models (such as the Hodgkin-Huxley model or related models) on the wire-frame neuron geometry. Moreover, while the running simulation is fed to the VR environment in real time, the user can interact with the surface mesh and affect the simulation while it is running, via several methods outlined under controls. Analogously, different types of synapses can be placed, connecting two different neuron vertices, as well as clamps and evaluation/plotting windows. The overarching philosophy is that the virtual laboratory feels close to a real wet-lab, in the sense that neurons, synapses, clamps, etc. are places and manipulated via the user's hands, and the user can observe and experience the resulting biophysical activity in many ways.



A key goal of this paradigm is that is can help accelerate scientific discovery by providing an immediate and intuitive feedback to the user about how changes to the simulated system affect the system's behavior. This insight, obtained from the simple and fast models used for VISOR, can then enable the computational scientist to devise significantly more targeted (non-interactive) simulations on large-scale HPC clusters of more complex models. In addition, in the context of neuroscience, the immersed 3D environment provides a more intuitive way to navigate and comprehend complex neuron geometries than traditional visualizations on computer screens, helping to overcome possible language gaps between computational and experimental science.

The Neuro-VISOR software is fully open source, and it can be run in VR mode (currently developed for Oculus VR hardware), as well as in desktop mode (without VR headset). While the desktop version does not provide the 3D visualization and intuitive controls via the user's hands of the VR version, it does provide all of the same interactive real-time simulation capabilities.

Neuro-VISOR has been used at multiple venues, including teaching/education (undergraduate neuroscience course at Temple University), scientific conferences (Mid-Atlantic Numerical Analysis Day, Latest trends and insights into matrix theory, iterative methods, and preconditioning, Mathematical Opportunities in Digital Twins), research symposia, and public events (Philadelphia Start Talking Science Festival).

License

We use a modified version of the GNU LPGL v3. Our license can be found in LICENSE.txt

Research Team

This project is produced at the Center for Computational Mathematics and Modeling (C2M2) at Temple University.

Project Leads: Dr. Benjamin Seibold (Github), Dr. Gillian Queisser

Researchers and Developers: Zachary Miksis

Past Developers: Rujeko Chinomona

Code Documentation

Our code documentation is generated using Doxygen. The completeness of this documentation is dependent on code commenting, so there may be gaps and imperfections. If you notice issues with this documentation, please report it to seibold@temple.edu.

Other Code (Related to this project)

Stephan Grein (in) maintains a separate project for generating neuron grids, as well as a project containing additional custom attributes which proved useful during development of Neuro-VISOR.

Connect with Us

Inquiries about the project can be made to seibold@temple.edu.

Performance issues and code-related questions can be sent to miksis@temple.edu. We also encourage use of GitHub Issues.

An old blog goes into finer detail about some of our past and current code solutions. This blog contains interesting details about various areas of the project.

Use Guide

Recommended Hardware

- **Oculus Rift, Rift S, or Quest Head Mounted Display (HMD)** - This project is currently tested on the Oculus Rift S and the Oculus Quest. It was previously developed on the original Rift headset. This

project was developed using the Oculus SDK. We have not been able to run this project on other headsets, but have been working to make it more cross-compatible.

- We do not have rigorously tested minimum computer specs. We have listed our hardware setup below for convenience. Oculus has provided a list of VR-ready computers, but we do not necessarily endorse any computer recommended by them.

Dell Precision 5820 Tower

CPU Intel Xeon W-2125 GPU NVIDIA GeForce GTX 1080 RAM 32 GB

Recommended Software

Recommended for Running as a Standalone Application and in Unity Editor

- Windows 10 or 11 64-bit, or MacOS 64-bit - This program may be functional on other operating systems, but its performance is not guaranteed. The emulator functions have been tested on Ubuntu 16.04 LTS.
- Relevant HMD software and drivers

Recommended for Running in Unity Editor

- Unity 2019.4.26f1 (LTS)
- We use the Oculus Desktop 2.38.4 package
- Git version >= 2.7.0
- Git LFS version >= 2.10.0
- Note: Git users (versions < 2.23.0) should clone the repository by using `git lfs clone`. All other should use `git clone`. For users of older git versions this remedies the problem that every LFS versioned file will ask the user for their password.
- A pre-commit hook will block commits made with inappropriate versions of Git, Git LFS, or Unity. Users should install the hooks by calling `./install_git_hooks.sh` from the root directory after cloning.
    - git lfs: `git lfs env` in a terminal/console.
    - git: `git --version`
    - Unity: see UnityEditor

Starting the Program

Make sure that your HMD is set up and that you have gone through the first-time setup. Ideally you have tested several other programs on your headset before using this software.

Running as a Standalone Application

1. Download the latest build release.
2. Start the application by running Neuro-VISOR.exe.

3. Upon startup, the user is placed in a model of C2M2's lab. The application will detect a VR headset and launch in VR/keyboard emulator mode automatically.
4. Our control scheme is outlined below. Try moving around and looking at your hands.

If you wish to use your own neuron models, place desired neuron model files in `Neuro-VISOR_Data/StreamingAssets/NeuronalDynamics/Geometries`. Any neurons found in this directory will be available at runtime.

Running in Unity Editor

1. Clone project to any location.
2. Ensure the correct version of the Unity Editor is installed
3. Open project in Unity and open Assets/Scenes/MainScene
4. Place desired neuron cell files in `Assets/StreamingAssets/NeuronalDynamics/Geometries`. Any cells found in this directory will be available at runtime.
5. Start the application by pressing play in the editor.
6. Upon startup, the user is placed in a model of C2M2's lab. The application will detect a VR headset and launch in VR/keyboard emulator mode automatically.
7. Our control scheme is outlined below. Try moving around and looking at your hands.

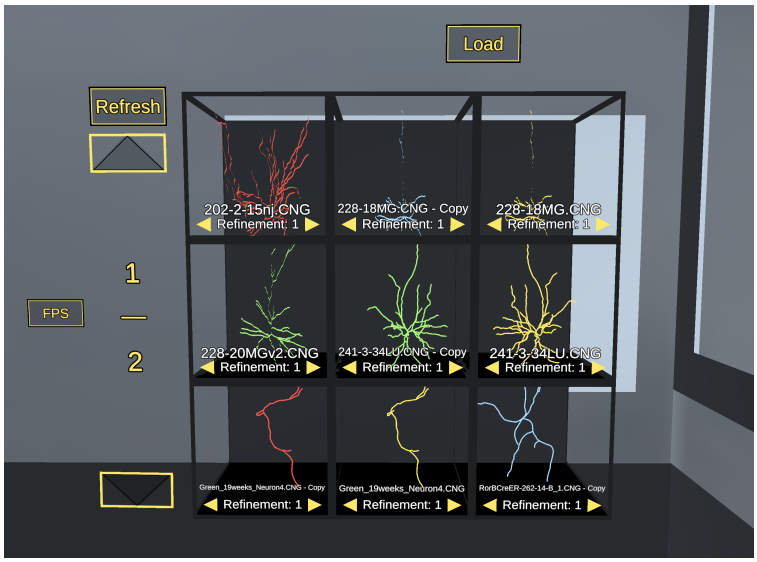
Controls

Oculus touch controller naming conventions can be found here. An instructional video demonstrating the controls can be found here. The term 'Hand Trigger' and 'Grip Trigger' are used analogously throughout the project, as are the terms 'Index Trigger' and 'Front Trigger'.

Action	Rift Touch Controllers	Keyboard/Mouse
Toggle raycasting	(P) Button one (A/X)	Always on
Interact	Raycast + (P/H) Index trigger	(P/H) Left mouse button
Grab	(H) Hand trigger	(H) Right mouse button
Reset Position	Grab + (P) Thumbstick	Raycast + 'X' key
Scale Object	Grab Ruler + Thumbstick up/down	Raycast Ruler + up/down arrow keys
Reset Scale	Grab Ruler + (P) Thumbstick	Raycast Ruler + 'R' key
Move Camera	Walk around!	WASD
Rotate Camera	Look around!	(H) Left-Ctrl + move mouse cursor
Quit	Oculus button	Escape key

Abbreviation	Meaning
(P)	Press button once
(H)	Hold button down

Selecting a Cell



1. A cell previewer stands against the whiteboard near the window. It attempts to render the 1D mesh of any neuron `.vrn` cell file archives found in `StreamingAssets/NeuronalDynamics/Geometries`. Six example cells are included with this repo.
2. Enable raycast mode. The hand with raycast mode enabled should be constantly pointing forward.
3. With raycast mode enabled, hover over a cell preview window by pointing at it. A blue guide line should be drawn between your pointer finger and the preview window. Continue hovering to see more information about the cell, or press the Interact button to load the cell and launch solve code. The guide line should turn orange while pressing/holding if in VR. The geometry should render in the middle of the room, scaled to fit within the room.

Simple Cell Interaction

Grabbing

The cell can be grabbed by hovering your hand over the 3D geometry and pressing the hand trigger on that hand's controller. Once grabbed, you can move and rotate cells freely.

Resizing

While grabbing a cell, hold the thumbstick up or down on the hand that is being used to grab the cell to resize the cell in world space. Note: this does not affect the environment of the solver code: the cell can be scaled freely in world space without affecting the stored vertex positions of the 1D or 3D meshes.

Board Info and Controls

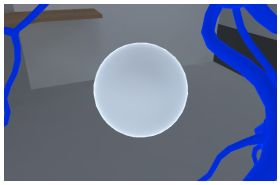
A large user interface is spawned upon selecting a cell. This board contains useful static information about the cell.

The board contains a color table that shows the current color gradient applied to the cell. The voltage value corresponding to different color values is diosplayed to the left of the color table. The user can change the value corresponding to the top or bottom of the color scale by hovering their finger cursor over either number and holding up/down on the joystick (or the up/down arrow key). The user can also change the color scheme by interacting with the arrows beneath the color table.

The board contains a subpanel for selecting the type of interaction. The user can select any of these toggles to change the effect of directly interacting with the surface. See direct, clamp, synapse and plot interactions for explanations of each.

The board also contains a play/pause button. The user can use this to pause solver code at the current time step. Beneath is displayed the current simulation time.

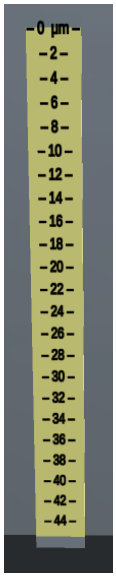
Pivot point object



A pivot point object is created when multiple neurons are present in the simulation. The pivot point object is represented as a white sphere, and is always located at the midpoint of all cell geometries. While grabbing the pivot point object, the cell geometries can be moved as a group.

When not grabbing the pivot point object, users can utilize the Scale Objects controls in order to rescale the objects within the simulation.

Ruler controls



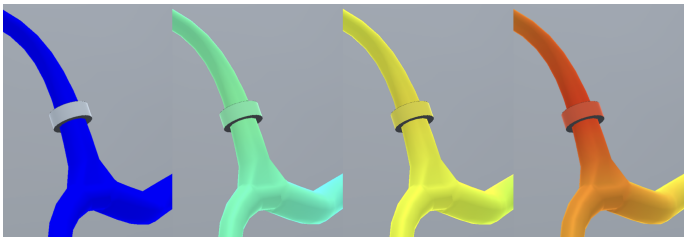
A ruler is spawned with every cell geometry. The ruler can be used to understand the length scale of the cell in its local space. While grabbing the ruler, resize it by moving the thumbstick up or down on the hand that is being used to grab the ruler.

The measurements on the ruler will adapt to the world-space size of the geometry so that it can always act as a translator between the size of the cell in the user's space and the local length scales of the neuron.

Direct cell interaction

With raycast mode enabled, point at the surface of the geometry. A blue line should be drawn between your pointer finger and the surface of the geometry. Tap the geometry from up close, or press the Interact button from a distance to directly alter simulation value at the nearest 1D vertex to the point of interaction. The guide line should turn orange upon pressing, and the surface of the geometry should change color to reflect the affected potential at the nearest 1D vertex on the geometry.

Clamp interaction



Clamps can be used to continuously alter the value of a single vertex on the 1D mesh. From the left to the right in the image above, there is a clamp that is off and then increasing in applied voltage.

Enable Clamp Mode

With the cell loaded and raycast mode enabled, press the `Clamp Mode` button on the whiteboard. "Finger clamps" should appear on both of the user's pointer fingers whilst raycasting. The finger clamps should appear as cylinders on the user's pointer finger.

With clamp mode enabled, interacting with the surface of the cell should place a cylindrical potential clamp on the cell near the point of interaction. The clamp will snap to the nearest 1D vertex to the 3D point of

interaction, and will effect that 1D vertex on the 1D mesh.

Toggling a clamp

Enable or disable the clamp by tapping it from up close or pressing the Interact button while pointing at it from a distance. The clamp is gray when it is disabled. It's color when enabled should reflect the clamp's current potential value.

Changing a clamp's power

While the clamp is enabled, the clamp's power can be changed by pointing at it, holding the Interact button, and moving the controller's joystick up or down.

Destroying a clamp

Clamps can be destroyed by holding the Interact button briefly while pointing at the clamp, and then releasing the Interact button.

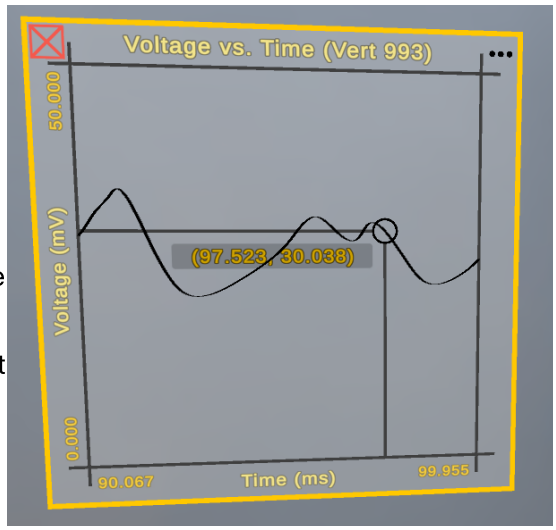
Group clamp controls, clamp highlighting

Users can place many clamps on the geometry. Clamps cannot be placed too near to each other, but there is no set limit to the number of clamps that can be placed.

In addition to each clamp being individually interactable, the "finger clamps" mentioned in Enable Clamp Mode are also interactable, and act as controllers for all existing clamps. Any control that can be done to a single clamp (toggle, alter power, destroy), can be done to all existing clamps at once by interacting with the finger clamp in the same manner.

The user can highlight all clamps with a red sphere to clarify their position to the user. This is particularly useful on complicated geometries with many clamps attached. While pointing at the finger clamp and holding the Interact button, hold down the hand trigger to highlight all existing spheres.

Point-plotter interaction



The user is able to spawn line graphs that are attached to specific 1D vertices on the neuron cell. These graphs will show the voltage at that 1D point over time.

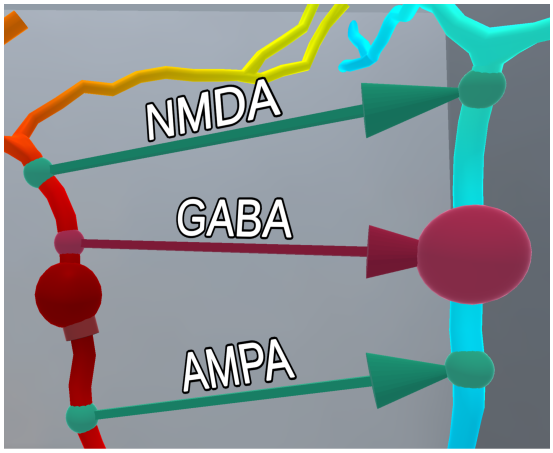
To spawn a graph panel, the user simply needs to toggle "Plot" on the board UI. At this point the user can raycast onto the surface of the mesh and press the index trigger (or click with the mouse) in order to spawn a graph attached to the nearest 1D vertex. Panels are grabbable and resizeable in the same way that the cell or the ruler is.

The user can hover over the graph-plane with their raycasting finger or the mouse in order to spawn a cursor at the point of hovering. The cursor will display the exact value of the graph at that point. The user can then press the index trigger (or click on the mouse) to lock the cursor at that position. Clicking again will free the cursor.

The number of samples in the graph can be altered by opening the "more info" panel (top-right corner of the graph), hovering over "number of samples", and holding up or down on the joystick (or holding the up or down arrows on the keyboard).

Synapse interaction

With the addition of principled synapse models into a multi-neuron network, one is now able to build a network of neurons and connect them via synapses, simulating transmission of electrical signals from one neuron to another. The project is equipped with three functional types of chemical synapse models found within the mammalian cerebellum. The implemented synapse models are the excitatory NMDA- and AMPA-receptor-based and the GABAergic inhibitory synapse. These are all modeled as outlined in J.S. Rothman, "Modeling Synapses" (2014).



1. To interact with synapses in the simulation, select and place neurons to be worked with into the main scene.
2. Left click in desktop mode or raycast in the VR environment onto the mode titled “Synapse” and select a vertex on the cell. The selected vertex will be the presynaptic membrane.
3. Next select a vertex on another neuron to be the postsynaptic site (It is possible to select a different vertex on the same cell to be the postsynaptic site, however the action potential propagation is clearer when there are synapses connecting two neurons).
4. The same synapse mode allows a user to switch between different types of synapse models, namely NMDA, AMPA, and GABA.
5. To switch between these modes, select (long-press) at the postsynaptic site, the mode will cycle through the implemented synaptic models.
6. The active synapse model is denoted by the synapse color displayed by the pre and postsynaptic sites as well as the arrow that points in the direction from these two membranes.
7. Note that the to distinguish between synaptic types, the GABA receptor in our simulation is colored red and the NMDA and AMPA receptors are colored green, each given a label above the arrow. The different synapse models, combined with the neuron signaling, represent the key ingredients needed to build micro-circuits with realistic signal processing capabilities.
8. Multiple synaptic connections can be placed on any vertex
9. Deleting one synapse connection triggers deletion of all other synapse connections associated with the vertex

Known Issues Log

- The programs freezes while a new neuron loads in
- Adding in a neuron while the simulation is paused, makes it appear white and buggy (resuming the simulation will fix this)
- Changing the color scale introduces various bugs
- Synapses may occasionally break when placed too quickly or too close together; needs more testing to determine exact cause
- The Cell Previewer's page scrolling buttons disappear when attempting to add a second Neuron into the simulation, making it impossible to load Neurons from different pages

- Movement of all neurons as a group configuration is not implemented on the Desktop version
- Pressing "close cell" may result in immediately loading another neuron if the cell previewer loads behind the "close cell" button after the simulation ends

Branches

Master: The latest stable version of the project

Development: The active beta, contains the latest features but at a higher risk of bugs

Changelog

2.6.1

Contributors: Adam Marx, Aidan Ross, Zachary Miksis

- Correction of performance issues in v2.6.0.
- Reorganization of numerical time stepping implementation.

2.6.0

Contributors: Adam Marx, Aidan Ross, Zachary Miksis

- New neurons are now loaded with a random rotation around the vertical axis and placed in the next "best" position relative to the position of existing neurons, favoring the center of the room.
- Available synapses now include NMDA, GABA, and AMPA, with visual labeling to indicate current type.
- Code improvements made to the synapse implementation to allow more structured addition of additional synapse models by power users.

2.5.0

Contributors: Malvin Prifti, Brandon Hugger, Rujeko Chinomona, Zachary Miksis

- Implemented a Pivot Point object which appears as a white sphere when multiple neurons are loaded
- Rescaling neurons is smoother, and now rescales them around the Pivot Point object
- Users can grab the Pivot Point object in VR to move all neurons around as a group
- Cell Previewer now has multiple pages and the ability to load new .vrn files while the simulation is running
- Presynaptic vertices now use a pink material until the user places a corresponding postsynaptic vertex
- Minor improvements to the way objects are generated within the simulation

2.1.1

Contributors: Brandon Calia, Rujeko Chinomona

Beta Testers: Amy Ollomani, Shreya Shah

- Multiple synaptic connections can be placed on the same vertex

- If a user deletes a synapse pair that has a pre or postsynaptic connection on a vertex that also contains other synapses, all involved synapses will be deleted
- Updates to some necessary packages

2.1.0

Contributors: Rujeko Chinomona, Craig Fox (in), Calin Pescaru, Garret Quallet, James Rosado (in), Wayne Zheng.

- Both excitatory and inhibitory synapses can now be added
- Synapses accutarely model real-world behavior
- Simulation configurations can be saved and loaded in
- Various performance improvements and consistence improvements were made

2.0.0

Contributors: Michael Bennett, Rujeko Chinomona, Craig Fox (in), James Rosado (in), Jacob Wells (in), Noah Williams, Wayne Zheng.

Beta Testers: Calin Pescaru, Garret Quallet

- More than one neuron can now be added
- Synapses can now be added **NOTE:** This is still a proof of concept feature and is not physically accurate
- Significant performance improvements were made
- Desktop mode now has all the features of VR
- MacOS is now supported

1.0.1

- Fixes group clamp control issue

1.0.0

Contributors: Craig Fox (in), Stephan Grein (in), Bogdan Nagirniak, James Rosado (in), Jacob Wells (in), Noah Williams.

Beta Testers: Marcus Bingenheimer, Rouying Tang

- Allows the creation of one, physically-accurate neuron
- Designed for VR, but a limited capability desktop version exists
- The neuron can have a voltage directly applied to it, can be clamped with a continuous voltage, and can have point plotters displaying voltage over time applied to it
- A ruler provides a length reference point
- A board displays cell information, controls the controls, sets the color information, and pauses, resumes, and quits the simulation